Python Activity-Based Learning

10 Case Studies • Algorithms • Line-by-Line Flowcharts • Pseudocode • Python Code • Test Cases

Updated version: all original case-study content is retained, with every flowchart rewritten as numbered, line-by-line steps for easier reading.

Case Study 1: University Scholarship Allocation

1. Problem Statement

Determine whether students qualify for a scholarship using aggregate marks, minimum subject marks, attendance, and family income. Rank eligible students by aggregate marks.

2. Objectives

- Check all scholarship eligibility conditions.
- Use lists and loops to examine multiple applicants.
- Rank eligible applicants by marks, with higher attendance as the tie-breaker.

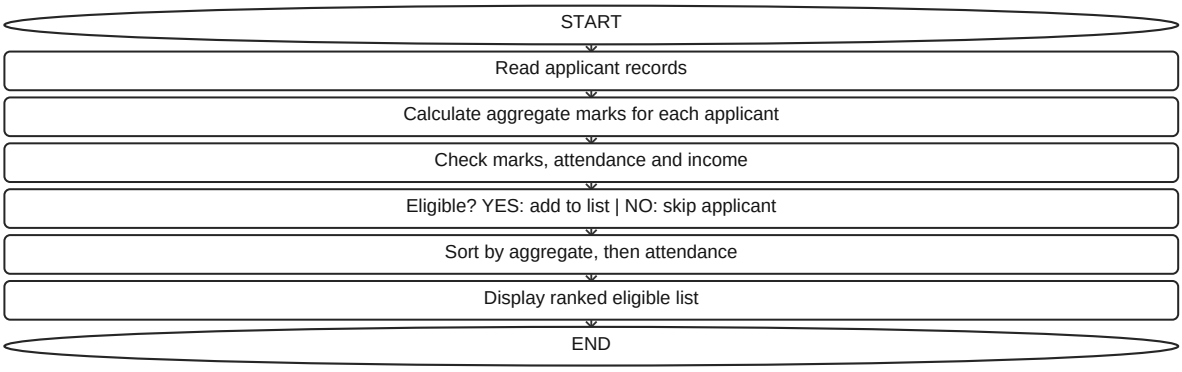
3. Assumptions

- Aggregate marks and attendance are percentages from 0 to 100.
- Each applicant has three subject marks.
- Eligibility requires aggregate >= 80, every subject >= 70, attendance >= 75, and annual family income <= ■300000.

4. Algorithm

- Start.
- Read applicant records.
- For each applicant, calculate the aggregate from the three subject marks.
- Check aggregate, subject, attendance, and income conditions.
- Store eligible applicants in a list.
- Sort by aggregate descending, then attendance descending.
- Display the ranked list and stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  eligible = empty list
  FOR each applicant
    aggregate = average(subject marks)
    IF aggregate >= 80 AND all subject marks >= 70 AND attendance >= 75 AND income <= 300000
      add applicant to eligible
  SORT eligible by aggregate descending, attendance descending
  DISPLAY eligible
END
```

7. Python Source Code

```
applicants = [
    ("Asha", [88, 91, 84], 92, 250000),
    ("Ravi", [78, 85, 82], 80, 200000),
    ("Meena", [90, 87, 89], 76, 280000),
    ("Kiran", [95, 92, 90], 88, 450000)
]
```

```
eligible = []
for name, marks, attendance, income in applicants:
    aggregate = sum(marks) / len(marks)
    if aggregate >= 80 and min(marks) >= 70 and attendance >= 75 and income <= 300000:
        eligible.append((name, aggregate, attendance))

eligible.sort(key=lambda student: (student[1], student[2]), reverse=True)
print("Scholarship Ranking")
if not eligible:
    print("No eligible applicants")
else:
    for rank, student in enumerate(eligible, start=1):
        print(f"{rank}. {student[0]} - Marks: {student[1]:.2f}%, Attendance: {student[2]}%")
```

8. Test Cases

Test input/scenario	Expected result
Asha: marks 88,91,84; attendance 92; income 250000	Eligible
Ravi: marks 78,85,82; attendance 80; income 200000	Not eligible: one subject below 70? No; aggregate 81.67, actually eligible
Kiran: income 450000	Not eligible: income exceeds limit

9. Output Screenshot

Run `solution.py` in Python. A sample console output is included as `output.png`; for interactive programs, enter the test inputs listed above.

10. Conclusion

The program applies multiple conditions to shortlist applicants and sorts eligible students consistently.

Sample Output

Sample console output (illustrative)

```
> Scholarship Ranking
> 1. Meena - Marks: 88.67%, Attendance: 76%
> 2. Asha - Marks: 87.67%, Attendance: 92%
```

Case Study 2: Progressive Electricity Billing

1. Problem Statement

Calculate a household's electricity bill using progressive consumption slabs, a fixed charge, and a rebate for low consumption.

2. Objectives

- Apply slab-wise calculations.
- Use conditionals and arithmetic operators.
- Print an itemized bill.

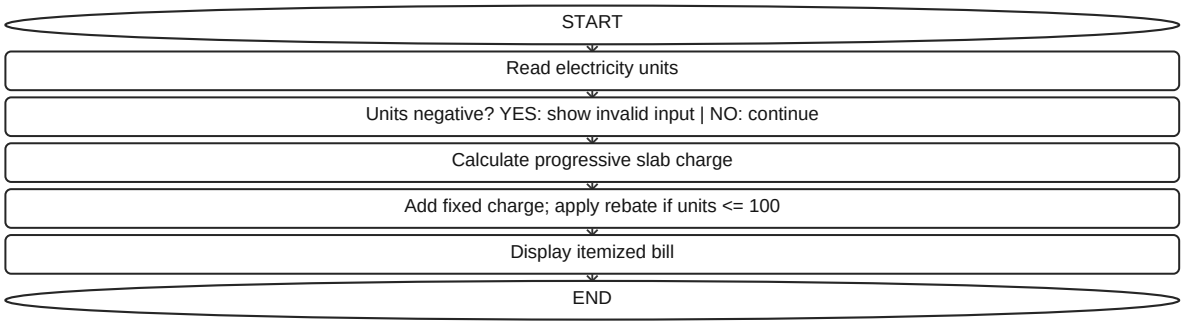
3. Assumptions

- First 100 units cost ■2 per unit; next 100 cost ■3; units above 200 cost ■5.
- Fixed charge is ■50.
- Consumption of 100 units or less receives a ■20 rebate.
- Consumption cannot be negative.

4. Algorithm

- Start and read units.
- Reject negative units.
- Calculate energy charge slab by slab.
- Add ■50 fixed charge.
- If units <= 100, subtract ■20 rebate.
- Display bill and stop.

5. Flowchart



6. Pseudocode

```
BEGIN
  READ units
  IF units < 0: DISPLAY invalid input
  ELSE
    calculate slab charge
    bill = slab charge + 50
    IF units <= 100: bill = bill - 20
    DISPLAY bill
END
```

7. Python Source Code

```
units = int(input("Enter electricity units: "))
if units < 0:
    print("Invalid units")
else:
    if units <= 100:
        energy_charge = units * 2
    elif units <= 200:
        energy_charge = 100 * 2 + (units - 100) * 3
```

```
else:
    energy_charge = 100 * 2 + 100 * 3 + (units - 200) * 5

fixed_charge = 50
rebate = 20 if units <= 100 else 0
total = energy_charge + fixed_charge - rebate

print("Energy charge: Rs.", energy_charge)
print("Fixed charge: Rs.", fixed_charge)
print("Rebate: Rs.", rebate)
print("Total bill: Rs.", total)
```

8. Test Cases

Test input/scenario	Expected result
Input 80	Energy 160; fixed 50; rebate 20; total ■190
Input 150	Energy 350; fixed 50; rebate 0; total ■400
Input -4	Invalid units

9. Output Screenshot

Run `solution.py` in Python. A sample console output is included as `output.png`; for interactive programs, enter the test inputs listed above.

10. Conclusion

The program calculates electricity charges fairly by applying progressive slabs and a low-use rebate.

Sample Output

```
Sample console output (illustrative)

> Enter electricity units: 80
> Energy charge: Rs. 160
> Fixed charge: Rs. 50
> Rebate: Rs. 20
> Total bill: Rs. 190
```

Case Study 3: Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate gross salary from basic pay, allowance, overtime, and incentive, then apply a simple deduction rate and print a salary statement.

2. Objectives

- Calculate gross and net salary.
- Use variables, arithmetic, and formatted output.
- Validate basic pay.

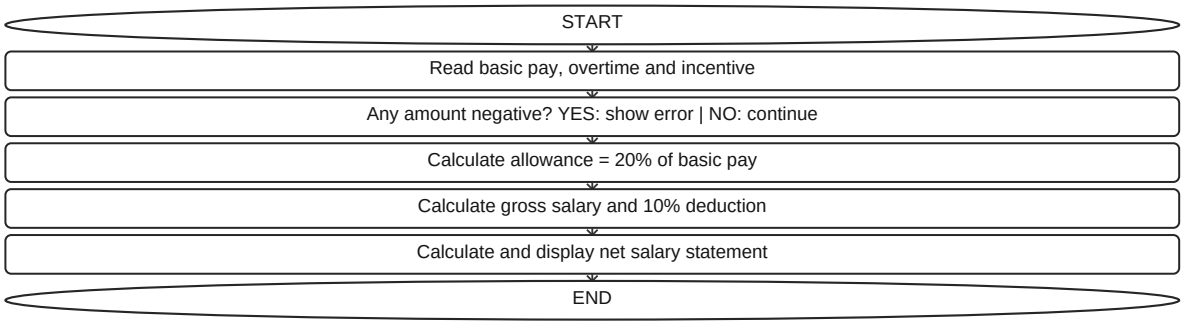
3. Assumptions

- Allowance is 20% of basic pay.
- Overtime pay and incentive are entered by the user.
- Deduction is 10% of gross salary.
- All entered amounts must be non-negative.

4. Algorithm

- Read basic pay, overtime pay, and incentive.
- Validate values.
- Calculate allowance and gross salary.
- Calculate deduction as 10% of gross.
- Calculate net salary.
- Display the salary statement.

5. Flowchart



6. Pseudocode

```
BEGIN
  READ basic, overtime, incentive
  IF any value < 0: DISPLAY invalid input
  ELSE
    allowance = basic * 0.20
    gross = basic + allowance + overtime + incentive
    deduction = gross * 0.10
    net = gross - deduction
    DISPLAY salary statement
END
```

7. Python Source Code

```
basic = float(input("Enter basic pay: Rs. "))
overtime = float(input("Enter overtime pay: Rs. "))
incentive = float(input("Enter incentive: Rs. "))

if basic < 0 or overtime < 0 or incentive < 0:
    print("Invalid salary amount")
else:
```

```
allowance = basic * 0.20
gross = basic + allowance + overtime + incentive
deduction = gross * 0.10
net = gross - deduction
print("\nSALARY STATEMENT")
print(f"Basic pay: Rs. {basic:.2f}")
print(f"Allowance: Rs. {allowance:.2f}")
print(f"Overtime: Rs. {overtime:.2f}")
print(f"Incentive: Rs. {incentive:.2f}")
print(f"Gross salary: Rs. {gross:.2f}")
print(f"Deduction: Rs. {deduction:.2f}")
print(f"Net salary: Rs. {net:.2f}")
```

8. Test Cases

Test input/scenario	Expected result
Basic 20000, overtime 1000, incentive 500	Allowance 4000; gross 25500; deduction 2550; net 22950
Basic -1, overtime 0, incentive 0	Invalid salary amount

9. Output Screenshot

Run `solution.py` in Python. A sample console output is included as `output.png`; for interactive programs, enter the test inputs listed above.

10. Conclusion

The program produces a clear salary statement and separates gross salary, deduction, and take-home pay.

Sample Output

```
Sample console output (illustrative)

> Enter basic pay: Rs. 20000
> Enter overtime pay: Rs. 1000
> Enter incentive: Rs. 500
> SALARY STATEMENT
> Gross salary: Rs. 25500.00
> Deduction: Rs. 2550.00
> Net salary: Rs. 22950.00
```

Case Study 4: Retail Billing with Discount Rules

1. Problem Statement

Calculate the bill for several products, apply a discount when the subtotal reaches a threshold, and add tax.

2. Objectives

- Use lists to store multiple products.
- Calculate subtotal, discount, tax, and final amount.
- Display an itemized invoice.

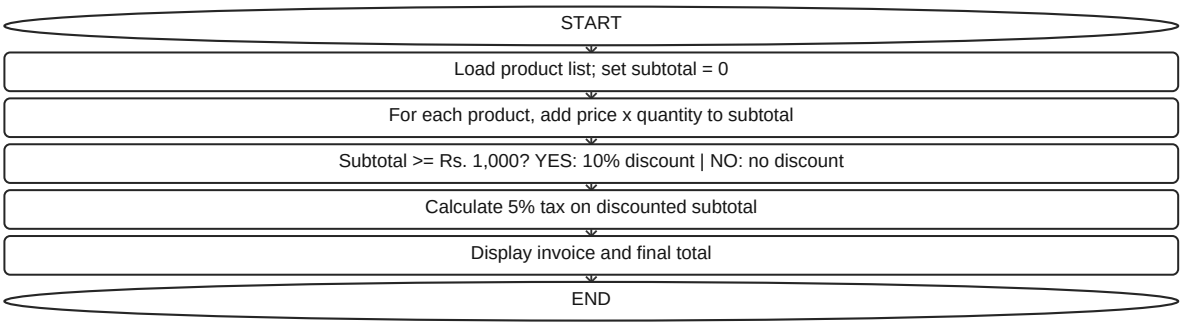
3. Assumptions

- Each product is stored as a tuple of name, price, and quantity.
- A 10% discount applies when subtotal is at least ■1000.
- Tax is 5% of the discounted subtotal.
- Prices and quantities are non-negative.

4. Algorithm

- Store product tuples in a list.
- Loop through products and calculate each line total.
- Add line totals to subtotal.
- Apply 10% discount if subtotal >= 1000.
- Calculate 5% tax on discounted subtotal.
- Display invoice totals.

5. Flowchart



6. Pseudocode

```
BEGIN
  subtotal = 0
  FOR each (name, price, quantity)
    line_total = price * quantity
    subtotal = subtotal + line_total
  IF subtotal >= 1000: discount = subtotal * 0.10 ELSE discount = 0
  taxable = subtotal - discount
  tax = taxable * 0.05
  total = taxable + tax
  DISPLAY totals
END
```

7. Python Source Code

```
products = [
    ("Notebook", 80, 3),
    ("Pen", 20, 5),
    ("Bag", 700, 1)
]
subtotal = 0
print("ITEMIZED INVOICE")
```

```
for name, price, quantity in products:
    line_total = price * quantity
    subtotal += line_total
    print(f"{name}: {quantity} x Rs.{price} = Rs.{line_total}")

discount = subtotal * 0.10 if subtotal >= 1000 else 0
taxable_amount = subtotal - discount
tax = taxable_amount * 0.05
total = taxable_amount + tax
print("Subtotal: Rs.", round(subtotal, 2))
print("Discount: Rs.", round(discount, 2))
print("Tax: Rs.", round(tax, 2))
print("Final total: Rs.", round(total, 2))
```

8. Test Cases

Test input/scenario	Expected result
Notebook 3x80, Pen 5x20, Bag 1x700	Subtotal ■1040; discount ■104; tax ■46.80; total ■982.80
Subtotal below ■1000	No discount; tax is 5% of subtotal

9. Output Screenshot

Run `solution.py` in Python. A sample console output is included as `output.png`; for interactive programs, enter the test inputs listed above.

10. Conclusion

The program automates an itemized bill and applies discount and tax rules in the correct order.

Sample Output

Sample console output (illustrative)

```
> ITEMIZED INVOICE
> Notebook: 3 x Rs.80 = Rs.240
> Pen: 5 x Rs.20 = Rs.100
> Bag: 1 x Rs.700 = Rs.700
> Subtotal: Rs. 1040
> Discount: Rs. 104.0
> Tax: Rs. 46.8
> Final total: Rs. 982.8
```

Case Study 5: Attendance Recovery Computation

1. Problem Statement

Calculate current attendance percentage and determine how many consecutive classes a student must attend to reach a target attendance percentage.

2. Objectives

- Calculate attendance percentage.
- Use a loop to find the minimum classes needed.
- Validate class counts.

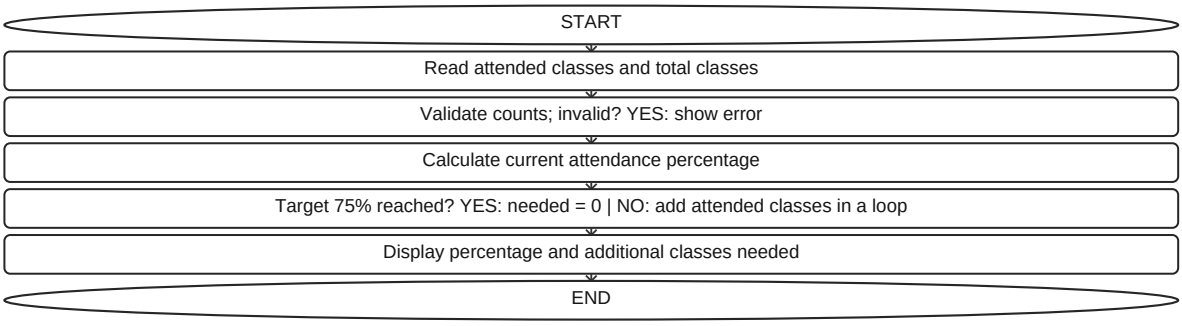
3. Assumptions

- Attended classes cannot exceed total classes.
- Target attendance is 75%.
- Future classes are all attended.

4. Algorithm

- Read attended and total classes.
- Validate counts.
- Calculate current percentage.
- If already at target, display zero additional classes.
- Otherwise add one attended class at a time until the percentage reaches 75%.
- Display result.

5. Flowchart



6. Pseudocode

```
BEGIN
  READ attended, total
  IF total <= 0 OR attended < 0 OR attended > total: DISPLAY invalid
  ELSE
    needed = 0
    WHILE (attended + needed) / (total + needed) < 0.75
      needed = needed + 1
    DISPLAY current percentage and needed
END
```

7. Python Source Code

```
attended = int(input("Classes attended: "))
total = int(input("Total classes held: "))
target = 75

if total <= 0 or attended < 0 or attended > total:
    print("Invalid attendance data")
else:
    current = attended / total * 100
```

```
needed = 0
while (attended + needed) / (total + needed) * 100 < target:
    needed += 1
print(f"Current attendance: {current:.2f}%")
print(f"Additional consecutive classes to attend: {needed}")
```

8. Test Cases

Test input/scenario	Expected result
Attended 60, total 100	Current 60%; need 60 additional classes to reach 75%
Attended 80, total 100	Current 80%; need 0 additional classes
Attended 12, total 0	Invalid attendance data

9. Output Screenshot

Run `solution.py` in Python. A sample console output is included as `output.png`; for interactive programs, enter the test inputs listed above.

10. Conclusion

The loop finds the minimum number of future classes required to meet the attendance threshold.

Sample Output

```
Sample console output (illustrative)

> Classes attended: 60
> Total classes held: 100
> Current attendance: 60.00%
> Additional consecutive classes to attend: 60
```

Case Study 6: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate deposits and withdrawals, update the account balance, reject invalid transactions, and show a transaction summary.

2. Objectives

- Use loops and conditionals for transaction processing.
- Maintain balance safely.
- Store transaction history in a list.

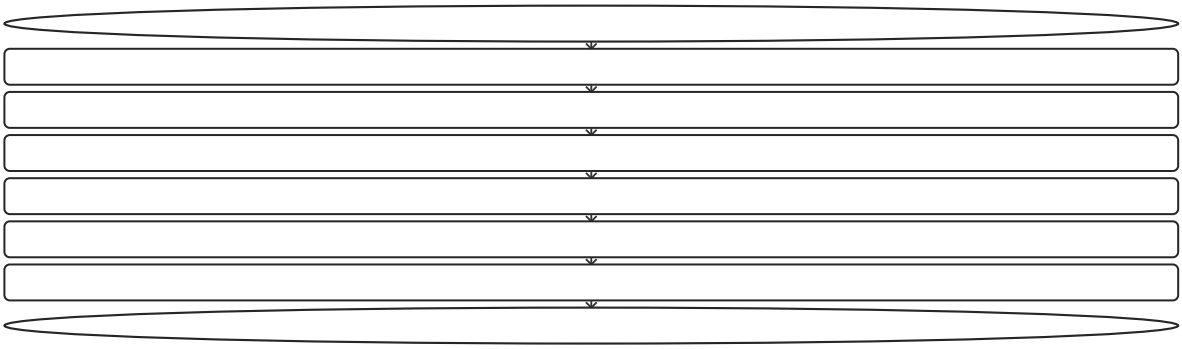
3. Assumptions

- Starting balance is ■5000.
- Transactions are entered as D for deposit or W for withdrawal, followed by a positive amount.
- A withdrawal cannot exceed the current balance.
- Enter X to finish.

4. Algorithm

- Set balance to ■5000 and create an empty history list.
- Repeatedly read transaction type and amount.
- Stop when type is X.
- Validate amount and transaction type.
- For valid deposits, add to balance; for valid withdrawals, check balance first.
- Record accepted transactions and display final summary.

5. Flowchart



6. Pseudocode

```
BEGIN
  balance = 5000
  history = empty list
  REPEAT
    READ type
    IF type == X: EXIT LOOP
    READ amount
    IF amount <= 0: reject
    ELSE IF type == D: balance = balance + amount; record
    ELSE IF type == W AND amount <= balance: balance = balance - amount; record
    ELSE: reject
  DISPLAY balance and history
END
```

7. Python Source Code

```
balance = 5000.0
history = []

while True:
    kind = input("Transaction (D=deposit, W=withdraw, X=exit): ").upper()
    if kind == "X":
```

```
        break
    if kind not in ("D", "W"):
        print("Invalid transaction type")
        continue
    amount = float(input("Amount: Rs. "))
    if amount <= 0:
        print("Amount must be positive")
    elif kind == "D":
        balance += amount
        history.append(("Deposit", amount, balance))
        print("Deposit successful")
    elif amount <= balance:
        balance -= amount
        history.append(("Withdrawal", amount, balance))
        print("Withdrawal successful")
    else:
        print("Insufficient balance")

print("\nTRANSACTION SUMMARY")
for kind, amount, remaining in history:
    print(f"{kind}: Rs.{amount:.2f}; Balance: Rs.{remaining:.2f}")
print(f"Final balance: Rs.{balance:.2f}")
```

8. Test Cases

Test input/scenario	Expected result
Deposit 1000; withdraw 2000; exit	Final balance ■4000
Withdraw 9000 from starting balance 5000	Rejected as insufficient balance; balance remains ■5000
Amount 0	Rejected because amount must be positive

9. Output Screenshot

Run `solution.py` in Python. A sample console output is included as `output.png`; for interactive programs, enter the test inputs listed above.

10. Conclusion

The simulation updates balances only for valid transactions and keeps a record of accepted operations.

Sample Output

```
Sample console output (illustrative)
> Transaction (D=deposit, W=withdraw, X=exit): D
> Amount: Rs. 1000
> Deposit successful
> Transaction (D=deposit, W=withdraw, X=exit): X
> TRANSACTION SUMMARY
> Deposit: Rs.1000.00; Balance: Rs.6000.00
> Final balance: Rs.6000.00
```

Case Study 7: Warehouse Inventory Reconciliation

1. Problem Statement

Compare opening stock, received stock, dispatched stock, and recorded closing stock to find discrepancies and replenishment needs.

2. Objectives

- Use tuples and lists for product records.
- Calculate expected closing stock.
- Identify discrepancies and shortages.

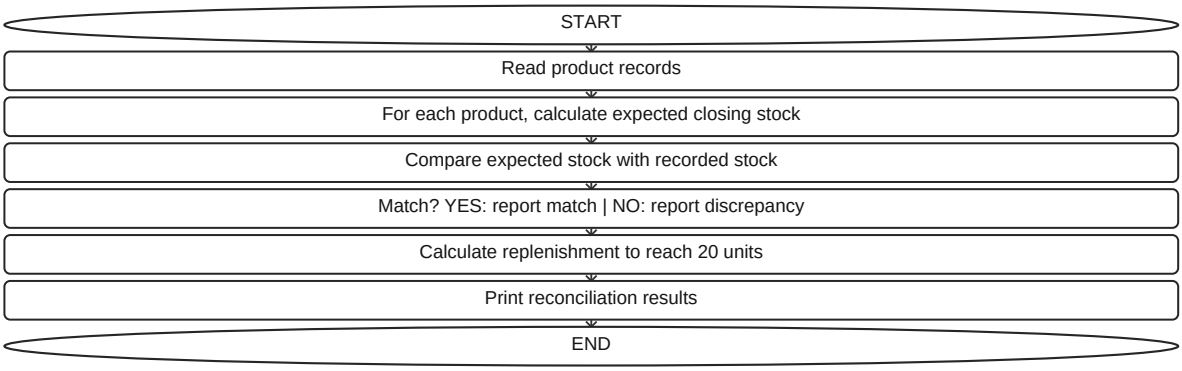
3. Assumptions

- A record contains product name, opening stock, received, dispatched, and recorded closing stock.
- All quantities must be non-negative.
- Target stock is 20 units for each product.

4. Algorithm

- Create a list of product tuples.
- For each product, calculate expected closing = opening + received - dispatched.
- Compare expected and recorded closing stock.
- Calculate replenishment needed to reach target stock of 20.
- Print results.

5. Flowchart



6. Pseudocode

```
BEGIN
  FOR each product tuple
    expected = opening + received - dispatched
    IF expected == recorded: status = Match ELSE status = Discrepancy
    replenish = maximum(0, 20 - expected)
    DISPLAY product, expected, status, replenish
END
```

7. Python Source Code

```
products = [
    ("Pens", 30, 15, 20, 25),
    ("Books", 40, 10, 25, 30),
    ("Folders", 12, 5, 3, 14)
]
target_stock = 20

print("INVENTORY RECONCILIATION")
for name, opening, received, dispatched, recorded in products:
    expected = opening + received - dispatched
    status = "Match" if expected == recorded else "Discrepancy"
    replenish = max(0, target_stock - expected)
    print(f"{name}: expected={expected}, recorded={recorded}, status={status}, replenish={replenish}")
```

8. Test Cases

Test input/scenario	Expected result
Pens: 30 opening, 15 received, 20 dispatched, 25 recorded	Expected 25; Match; replenish 0
Books: 40 opening, 10 received, 25 dispatched, 30 recorded	Expected 25; Discrepancy; replenish 0
Expected stock 14	Replenishment 6

9. Output Screenshot

Run `solution.py` in Python. A sample console output is included as `output.png`; for interactive programs, enter the test inputs listed above.

10. Conclusion

The program checks stock records against calculated quantities and highlights products needing correction or replenishment.

Sample Output

Sample console output (illustrative)

```
> INVENTORY RECONCILIATION
> Pens: expected=25, recorded=25, status=Match, replenish=0
> Books: expected=25, recorded=30, status=Discrepancy, replenish=0
> Folders: expected=14, recorded=14, status=Match, replenish=6
```

Case Study 8: Advanced Numeric Pattern Generator

1. Problem Statement

Generate a symmetric hollow diamond pattern for a positive odd width.

2. Objectives

- Practice nested loops and conditions.
- Generate aligned text patterns.
- Validate the requested width.

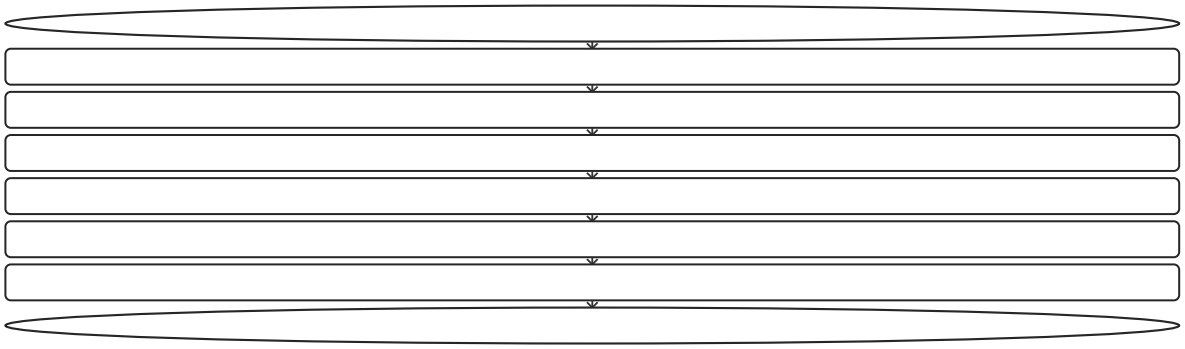
3. Assumptions

- The width must be a positive odd integer.
- The widest row contains the requested number of positions.
- The pattern uses asterisks at the outline and spaces inside.

4. Algorithm

- Read width.
- Reject values less than 1 or even values.
- Set half = width // 2.
- For each row from top to bottom, calculate leading spaces and outline positions.
- Print the upper and lower halves using nested loops.

5. Flowchart



6. Pseudocode

```
BEGIN
  READ width
  IF width < 1 OR width is even: DISPLAY invalid
  ELSE
    FOR row from 0 to width-1
      distance = min(row, width-1-row)
      leading_spaces = half - distance
      inner_spaces = 2 * distance - 1
      print leading spaces and outline stars
END
```

7. Python Source Code

```
width = int(input("Enter a positive odd width: "))
if width < 1 or width % 2 == 0:
    print("Width must be a positive odd number")
else:
    half = width // 2
    for row in range(width):
        distance = min(row, width - 1 - row)
        leading = half - distance
        print(" " * leading, end="")
        if distance == 0:
            print("*")
        else:
            print("*" + " " * (2 * distance - 1) + "*")
```

8. Test Cases

Test input/scenario	Expected result
Input 5	*; * *; * *; * *; * (alignment depends on leading spaces)
Input 4	Width must be a positive odd number
Input 1	*

9. Output Screenshot

Run `solution.py` in Python. A sample console output is included as `output.png`; for interactive programs, enter the test inputs listed above.

10. Conclusion

Nested loops and spacing calculations create a symmetric hollow diamond while validating dimensions.

Sample Output

```
Sample console output (illustrative)

> Enter a positive odd width: 5
>     *
>    * *
>   *   *
>    * *
>     *
```

Case Study 9: Student Marks Matrix Analysis

1. Problem Statement

Analyze student marks stored in a nested list, calculate each student's total and average, and find the highest-performing student.

2. Objectives

- Represent marks using a nested list.
- Calculate row-wise statistics.
- Identify the top performer.

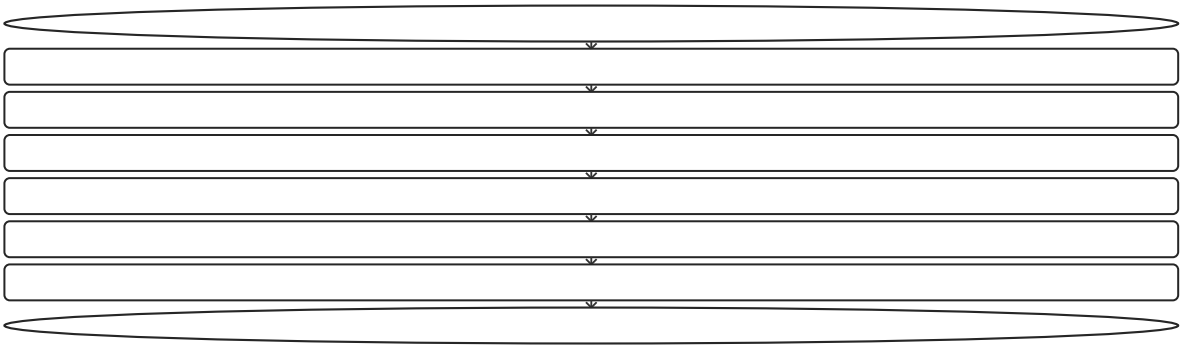
3. Assumptions

- Each row contains one student's marks in three subjects.
- Marks are from 0 to 100.
- Student names are stored in a separate list in the same order.

4. Algorithm

- Store student names and marks matrix.
- For each row, calculate total and average.
- Print each student's statistics.
- Track the highest total and print the top performer.

5. Flowchart



6. Pseudocode

```
BEGIN
  highest_total = -1
  FOR each student row
    total = sum(row)
    average = total / number of subjects
    DISPLAY student, total, average
    IF total > highest_total: update top student
  DISPLAY top student
END
```

7. Python Source Code

```
names = ["Asha", "Ravi", "Meena"]
marks = [
    [88, 91, 84],
    [76, 82, 79],
    [95, 92, 90]
]
top_index = 0
highest_total = -1

print("STUDENT MARKS REPORT")
for i in range(len(marks)):
    total = sum(marks[i])
    average = total / len(marks[i])
    print(f"{names[i]}: total={total}, average={average:.2f}")
    if total > highest_total:
        highest_total = total
```

```
        top_index = i
print(f"Top performer: {names[top_index]} with {highest_total} marks")
```

8. Test Cases

Test input/scenario	Expected result
Asha 88,91,84	Total 263; average 87.67
Meena 95,92,90	Total 277; average 92.33; top performer
Three students with valid marks	One summary line per student and a top performer

9. Output Screenshot

Run `solution.py` in Python. A sample console output is included as `output.png`; for interactive programs, enter the test inputs listed above.

10. Conclusion

The program uses a nested list to summarize marks and determine the highest-performing student.

Sample Output

```
Sample console output (illustrative)

> STUDENT MARKS REPORT
> Asha: total=263, average=87.67
> Ravi: total=237, average=79.00
> Meena: total=277, average=92.33
> Top performer: Meena with 277 marks
```

Case Study 10: Tuple-Based Product Catalogue Audit

1. Problem Statement

Audit product records stored as tuples by checking prices and stock, calculating inventory value, and identifying items that need replenishment.

2. Objectives

- Store fixed product records in tuples.
- Use tuple unpacking in a loop.
- Validate records and calculate inventory value.

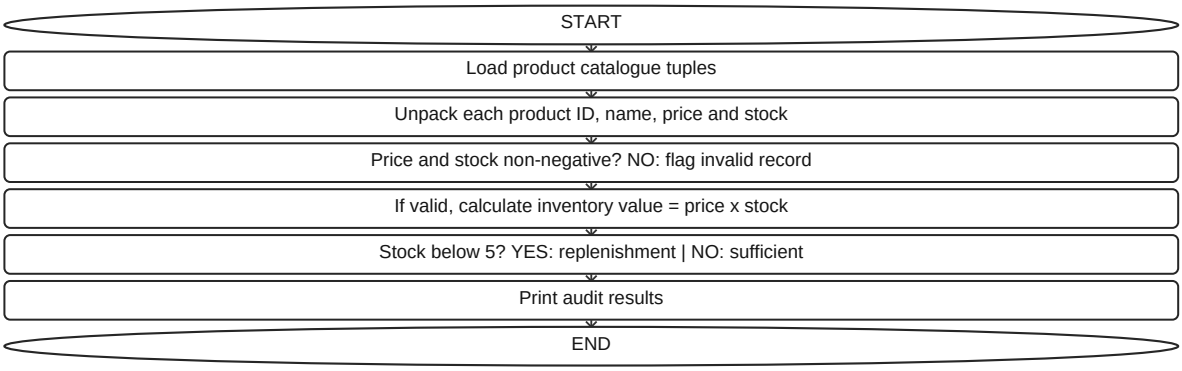
3. Assumptions

- Each tuple contains product ID, product name, price, and stock quantity.
- Price and stock must not be negative.
- Products with fewer than 5 units need replenishment.

4. Algorithm

- Create a list of product tuples.
- Unpack each tuple in a loop.
- Flag negative price or stock as invalid.
- For valid records, calculate price multiplied by stock.
- Flag stock below 5 for replenishment.
- Print audit results.

5. Flowchart



6. Pseudocode

```
BEGIN
  FOR each (id, name, price, stock)
    IF price < 0 OR stock < 0: DISPLAY invalid record
    ELSE
      value = price * stock
      IF stock < 5: status = replenish ELSE status = sufficient
      DISPLAY record and value
END
```

7. Python Source Code

```
products = [
    ("P101", "Notebook", 60.0, 20),
    ("P102", "Marker", 35.0, 3),
    ("P103", "Folder", 25.0, 0),
    ("P104", "Bottle", 120.0, 8)
]

print("PRODUCT CATALOGUE AUDIT")
for product_id, name, price, stock in products:
```

```
    if price < 0 or stock < 0:
        print(f"{product_id} {name}: Invalid record")
    else:
        value = price * stock
        status = "Replenishment required" if stock < 5 else "Stock sufficient"
        print(f"{product_id} {name}: value=Rs.{value:.2f}, stock={stock}, {status}")
```

8. Test Cases

Test input/scenario	Expected result
P101 price 60, stock 20	Inventory value ■1200; stock sufficient
P102 price 35, stock 3	Inventory value ■105; replenishment required
Negative price or stock	Invalid record

9. Output Screenshot

Run `solution.py` in Python. A sample console output is included as `output.png`; for interactive programs, enter the test inputs listed above.

10. Conclusion

The program demonstrates tuple unpacking and validates catalogue data while identifying low-stock products.

Sample Output

```
Sample console output (illustrative)

> PRODUCT CATALOGUE AUDIT
> P101 Notebook: value=Rs.1200.00, stock=20, Stock sufficient
> P102 Marker: value=Rs.105.00, stock=3, Replenishment required
> P103 Folder: value=Rs.0.00, stock=0, Replenishment required
> P104 Bottle: value=Rs.960.00, stock=8, Stock sufficient
```